

Rhilo Sotto

COMPE470

8 November 2024

Assignment 4

Modules

Programmable Square-Wave Generator

```
module SquareWaveGenerator #(parameter [2:0] M = 4, N = 2) (input CLK, RST, output wave);

localparam CLK_FREQ = 1;
integer counter = 0;
reg sq_wave_reg = 0;
assign wave = sq_wave_reg;

always @(posedge CLK) begin
    if (RST) begin
        counter <= 0;
        sq_wave_reg <= 1'b0;
    end
    else begin
        if (counter == 0) begin
            sq_wave_reg <= ~sq_wave_reg;
            if (sq_wave_reg == 0) begin
                counter <= M*CLK_FREQ - 1;
            end
            else begin
                counter <= N*CLK_FREQ - 1;
            end
        end
        else
            counter <= counter - 1;
        end
    end
end
endmodule
```

LFSR_A

```
module LFSR_A(input CLK, PRST, output reg [3:0] Q);

always @(posedge CLK) begin
    if (PRST) Q <= 4'b1111;
    else Q <= {Q[2], Q[1], Q[0]^Q[3], Q[3]};
end

endmodule
```

LFSR_B

```
module LFSR_B(input CLK, PRST, serial_in, output reg [3:0] Q);

always @(posedge CLK) begin
    if (PRST) Q <= 4'b1111;
    else Q <= {Q[2], Q[1], Q[0]^Q[3], Q[3]^serial_in};
end

endmodule
```

Testbenches

Programmable Square-Wave Generator

```
`timescale 1ns / 1ns

module tb_SquareWaveGenerator;

    localparam CLK_PERIOD = 10;
    reg CLK = 0;
    always #(CLK_PERIOD / 2) begin
        CLK <= ~CLK;
    end

    reg RST = 1;
    wire out;

    // instantiate DUT
    SquareWaveGenerator #(.M(3), .N(5)) SQG(.CLK(CLK), .RST(RST), .wave(out));

    always @(posedge CLK) begin
        @(posedge CLK);
        RST = 0;
        repeat(200) @(posedge CLK);

    $finish;
    end

endmodule
```

LFSR_A

```
`timescale 1ns / 1ns
```

```
module tb_LFSR_A;
```

```
localparam CLK_PERIOD = 10;
```

```
reg CLK = 0;
```

```
always #(CLK_PERIOD / 2) begin
```

```
    CLK <= ~CLK;
```

```
end
```

```
reg RST = 1;
```

```
wire [3:0] out;
```

```
// instantiate DUT
```

```
LFSR_A LA(.CLK(CLK), .PRST(RST), .Q(out));
```

```
always @(posedge CLK) begin
```

```
    @(posedge CLK);
```

```
    RST = 0;
```

```
    repeat(30) @(posedge CLK);
```

```
$finish;
```

```
end
```

```
endmodule
```

LFSR_B

```
`timescale 1ns / 1ns
```

```
module tb_LFSR_B;
```

```
    localparam CLK_PERIOD = 10;
```

```
    reg CLK = 0;
```

```
    always #(CLK_PERIOD / 2) begin
```

```
        CLK <= ~CLK;
```

```
    end
```

```
    reg RST = 1;
```

```
    reg in;
```

```
    wire [3:0] out;
```

```
    // instantiate DUT
```

```
    LFSR_B LB(.CLK(CLK), .PRST(RST), .serial_in(in), .Q(out));
```

```
    always @(posedge CLK) begin
```

```
        @(posedge CLK);
```

```
        RST = 0;
```

```
        in = 1;
```

```
        @(posedge CLK);
```

```
        in = 0;
```

```
        @(posedge CLK);
```

```
        in = 1;
```

```
        @(posedge CLK);
```

```
        in = 1;
```

```
        @(posedge CLK);
```

```
        in = 0;
```

```
        @(posedge CLK);
```

```
        in = 0;
```

```
        @(posedge CLK);
```

```
        in = 1;
```

```
        @(posedge CLK);
```

```
        in = 0;
```

```
        @(posedge CLK);
```

```
        in = 0;
```

```
        @(posedge CLK);
```

```
        in = 0;
```

```
        @(posedge CLK);
```

```
        in = 0;
```

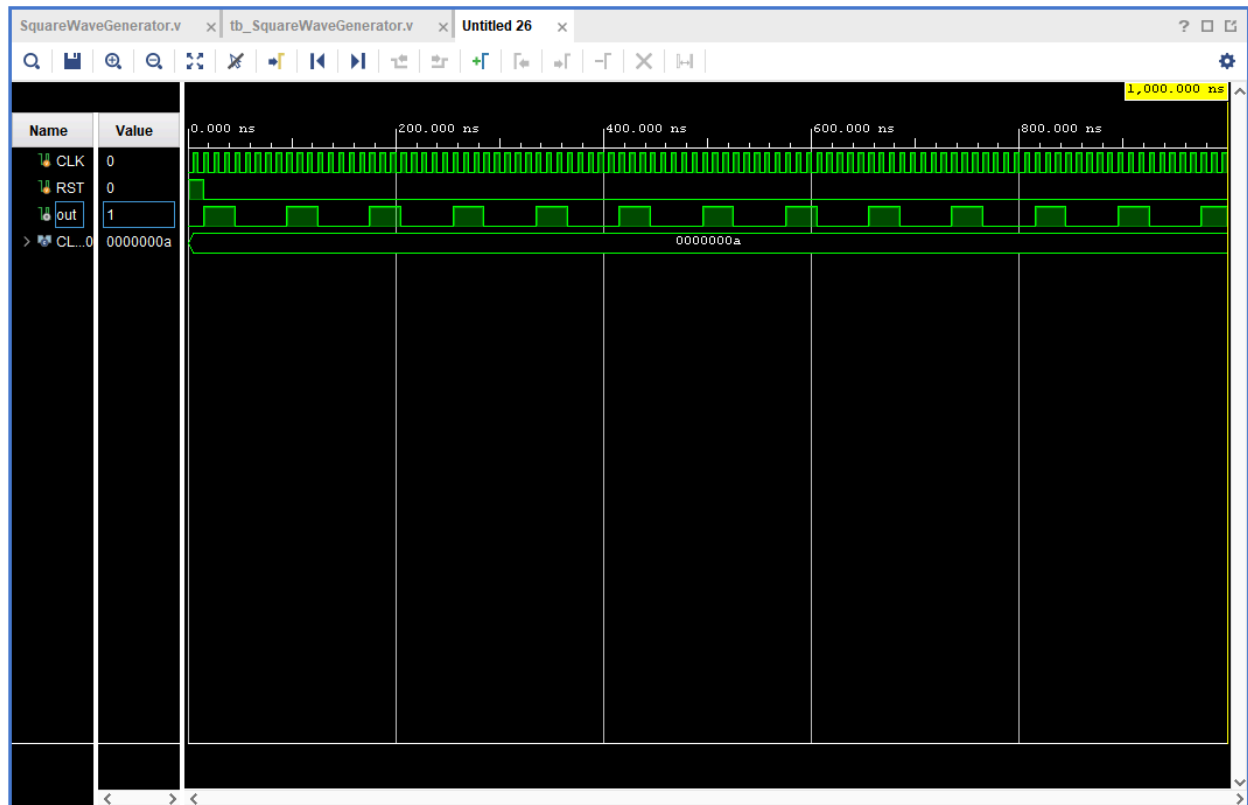
```
        @(posedge CLK);
```

```
$finish;  
end
```

```
endmodule
```

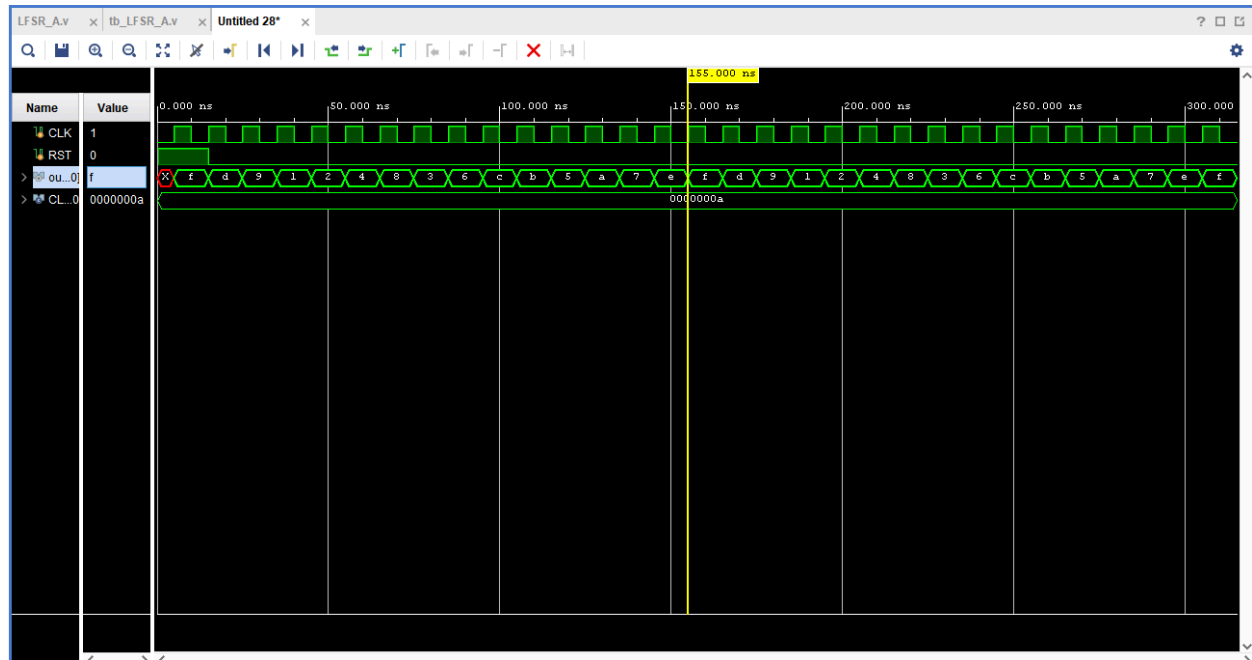
Waveforms

Programmable Square-Wave Generator



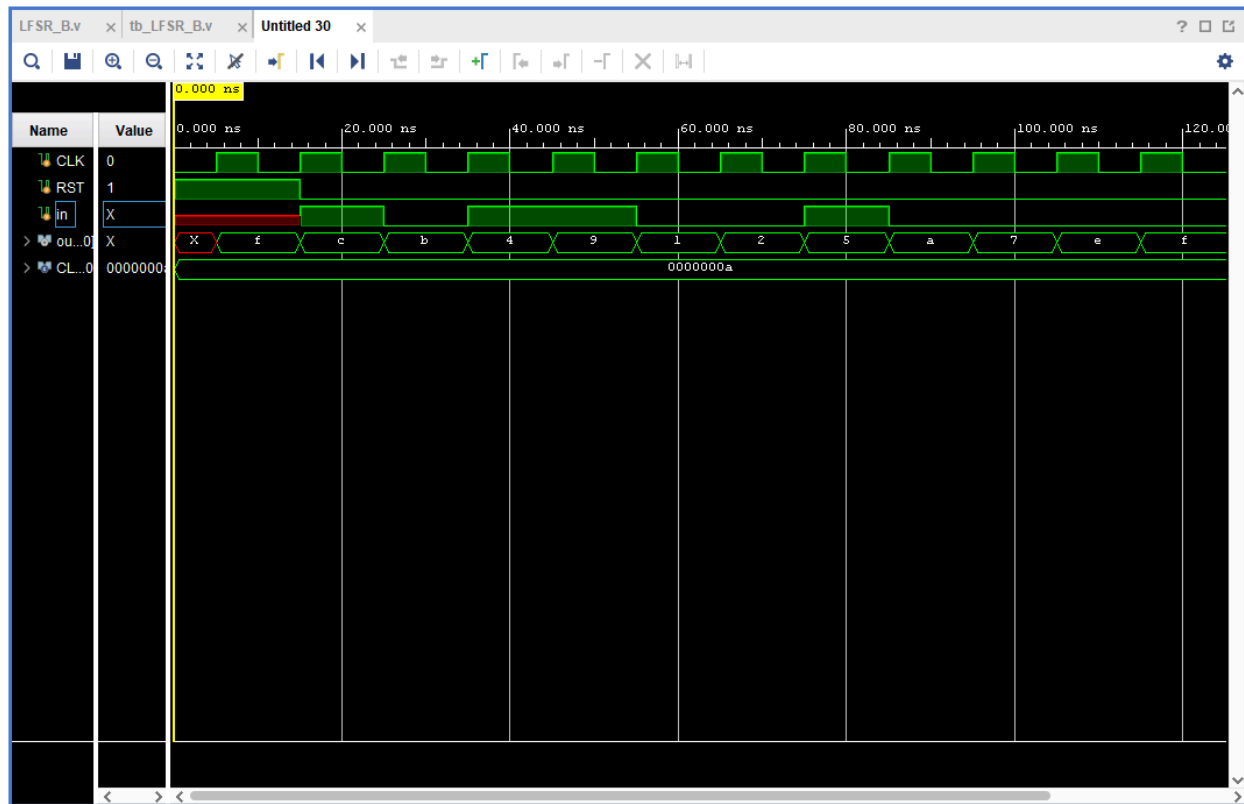
Corresponding to the programmable square-wave modules and testbench, the output is as expected.

LFSR_A



Corresponding to the LFSR in Q2a modules and testbench, the output is as expected.

LFSR_B



Corresponding to the LFSR in Q2b modules and testbench, the output is as expected.